AI ASSISTANT CODING

ASSIGNMENT -8.1

Name: G.V.N Hasini

HT. No: 230351099

Batch: 02

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.
- Requirements:
  - Password must have at least 8 characters.
  - Must include uppercase, lowercase, digit, and special character.
  - Must not contain spaces.

Example Assert Test Cases:

assert is_strong_password("Abcd@123") == True

assert is_strong_password("abcd123") == False

assert is_strong_password("ABCD@1234") == True

Expected Output #1:

Password validation logic passing all AI-generated test cases

assg_08.py

```python
def password_check(password):
    if len(password) < 8:
        return False
    if not any(char.isupper() for char in password):
        return False
    if not any(char.islower() for char in password):
        return False
    if not any(char.isdigit() for char in password):
        return False
    if not any(char in "!@#$%^&*()-_=+[]{}|;:'\",.<>?/" for char in password):
        return False
    return True
```

```python
assg_08.py > ...
def password_check(password):
    if len(password) < 8:
        return False
    if not any(char.isupper() for char in password):
        return False
    if not any(char.islower() for char in password):
        return False
    if not any(char.isdigit() for char in password):
        return False
    if not any(char in "!@#$%^&*()-_=+[]{}|;:'\",.<>?/" for char in password):
        return False
    return True
#assert testcases
assert password_check("Password123!") == True
assert password_check("pass") == False
assert password_check("PASSWORD123") == False
assert password_check("password123") == False
assert password_check("Password") == False
assert password_check("Password123") == False
assert password_check("Password!") == False
assert password_check("12345678") == False
assert password_check("!@#$%^&*") == False
print("All test cases passed!")
```

OUTPUT:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/python.exe c:/
All test cases passed!
```

<u>Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)</u>

- Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.
- Requirements:
    - Classify numbers as Positive, Negative, or Zero.
    - Handle invalid inputs like strings and None.
    - Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

assert classify_number(10) == "Positive"

assert classify_number(-5) == "Negative"

assert classify_number(0) == "Zero"

Expected Output #2:

- Classification logic passing all assert tests.

```python
def classify_number(n):
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"
assert classify_number(10) == "Positive"
```

```python
def classify_number(n):
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
print("All test cases passed!")
```

OUTPUT:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/python.exe c:/
All test cases passed!
```

<u>Task Description #3 (Anagram Checker – Apply AI for String Analysis)</u>

- Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.
- Requirements:
    - Ignore case, spaces, and punctuation.
    - Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

```python
def is_anagram(str1, str2):
    return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
```

```python
def is_anagram(str1, str2):
    return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("The eyes", "They see") == True
assert is_anagram("Astronomer", "Moon starer") == True
assert is_anagram("Conversation", "Voices rant on") == True
print("All test cases passed!")
```

OUTPUT:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/python.exe c:/
All test cases passed!
```

<u>Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)</u>

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
- Methods:
    - add_item(name, quantity)
    - remove_item(name, quantity)
    - get_stock(name)

Example Assert Test Cases:

inv = Inventory()

inv.add_item("Pen", 10)

assert inv.get_stock("Pen") == 10

inv.remove_item("Pen", 5)

assert inv.get_stock("Pen") == 5

inv.add_item("Book", 3)

assert inv.get_stock("Book") == 3

Expected Output #4:

- Fully functional class passing all assertions.

```python
class inventory:
    def __init__(self):
        self.items = {}
    def add_item(self, item, quantity):
        if item in self.items:
            self.items[item] += quantity
        else:
            self.items[item] = quantity
    def remove_item(self, item, quantity):
        if item in self.items and self.items[item] >= quantity:
            self.items[item] -= quantity
            if self.items[item] == 0:
                del self.items[item]
        else:
            raise ValueError("Not enough items in inventory")
    def get_quantity(self, item):
        return self.items.get(item, 0)
```

OUTPUT:

```
All test cases passed!
```

```python
class inventory:
    def __init__(self):
        self.items = {}

    def add_item(self, item, quantity):
        if item in self.items:
            self.items[item] += quantity
        else:
            self.items[item] = quantity

    def remove_item(self, item, quantity):
        if item in self.items:
            if self.items[item] >= quantity:
                self.items[item] -= quantity
                if self.items[item] == 0:
                    del self.items[item]
            else:
                raise ValueError("Not enough quantity to remove")
        else:
            raise ValueError("Item not found in inventory")

    def get_stock(self, item):
        return self.items.get(item, 0)
inv = inventory()

inv.add_item("apple", 10)
assert inv.get_stock("apple") == 10

inv.add_item("banana", 5)
assert inv.get_stock("banana") == 5

inv.remove_item("apple", 3)
assert inv.get_stock("apple") == 7

inv.remove_item("banana", 5)
assert inv.get_stock("banana") == 0

print("All test cases passed!")
```

OUTPUT:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/python.exe c:/
All test cases passed!
```

<u>Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)</u>

- Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.
- Requirements:
    - Validate "MM/DD/YYYY" format.
    - Handle invalid dates.

- Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"

assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

```
def validate_and_formate_date(date_str):
    import re
    from datetime import datetime
    pattern = r'^\d{2}/\d{2}/\d{4}$'
        if not re.match(pattern, date_str):
            raise ValueError("Date must be in the format DD/MM/YYYY")
```

```
def validate_and_format_date(date_str):
    import re
    from datetime import datetime
    pattern = r'^\d{2}/\d{2}/\d{4}$'
    if not re.match(pattern, date_str):
        return "Invalid Date"
    try:
        date_obj = datetime.strptime(date_str, "%m/%d/%Y")
        return date_obj.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"
assert validate_and_format_date("12/31/2020") == "2020-12-31"
assert validate_and_format_date("31/12/2020") == "Invalid Date"
assert validate_and_format_date("02/30/2020") == "Invalid Date"
assert validate_and_format_date("01/01/2021") == "2021-01-01"
assert validate_and_format_date("13/01/2020") == "Invalid Date"
print("All test cases passed!")
```

OUTPUT:

```
All test cases passed!
```